AEROTRACE

Airport Baggage Recovery & Intelligent Transfer Orchestration

Senior Java Backend Engineer - Technical Case Study

Java 25 | Spring Boot Microservices | Kafka Streams | Kubernetes

Design and implement a fault-tolerant platform that tracks checked baggage across airport checkpoints, identifies missed connections, coordinates recovery actions and provides auditable operational intelligence.

Document type: Candidate assignment and evaluation specification

Language: English Target level: Senior / Staff-ready backend engineer

Submission: Git repository, working services, tests, environment manifests, dashboards and architecture documentation

1. Business Context and Problem Statement

AeroTrace operates baggage-handling integrations for multiple airlines and airports. Bags move through check-in, security, sorting, aircraft loading, transfer and collection. Checkpoint scanners emit observations, sometimes late, repeated or out of sequence. Missed connections must trigger recovery without assigning the same bag to multiple conflicting routes.

Develop a backend-only microservices system to ingest scan events, maintain a reliable baggage journey, assess connection risk, create recovery cases, reserve transport capacity and present searchable operational status. This is a fictional domain; no connection to actual airline systems is required.

2. Scope, Actors and Assumptions

- Actors: passenger, airline operator, airport dispatcher, recovery agent, and platform administrator.
- A bag belongs to one active itinerary at a time; itinerary contains one or more flight legs and minimum connection times.
- Airport timestamps are normalized to UTC, but original source timezone and device timestamp are retained.
- No real passenger identity is required: seed pseudonymous passenger and flight information.
- Do not implement aircraft control, physical conveyor control or real airline ticketing.

3. Required Technology Stack

Area	Mandatory technology
Core	Java 25, Spring Boot, Maven multi-module project
Platform	Eureka Server, Spring Cloud API Gateway, shared Common Module
Streaming	Apache Kafka, Kafka Streams (event time and state stores)
Storage	PostgreSQL, Liquibase, Redis, Elasticsearch
Security and contracts	Spring Security, JWT, RBAC, OpenAPI, Spring HTTP Interface (@HttpExchange)
Testing	JUnit 5, Mockito, Spring Boot Test, Testcontainers
DevOps	Docker / Compose, Kubernetes (Minikube), GitHub Actions, SonarQube
Observability	Micrometer / Prometheus, Grafana, Loki, Tempo (OpenTelemetry tracing)

4. System Architecture and Service Boundaries

Module / service	Responsibilities
discovery-server	Eureka service registration and discovery; development profile
api-gateway	Single external entry point; token verification, routing, rate limiting, correlation IDs
common-module	Versioned API error contracts, event envelopes, shared security primitives; no JPA entities
identity-service	Test users, roles, signed access tokens, key rotation and authorization policies
journey-service	Baggage registration, itinerary snapshots, scan ingestion API and authoritative journey state
risk-streams-service	Kafka Streams topology computing connection risk and scan anomalies
recovery-service	Recovery case state machine, handler assignment and action orchestration
capacity-service	Finite recovery cart/flight/bag-slot capacity with atomic reservations
search-service	Elasticsearch read models for journey/case search and reconciliation
notification-service	Idempotent in-app notifications and delivery status; external provider can be mocked

Each domain service owns its schema and Liquibase changelog. Cross-service data access must use events or a typed HTTP Interface client, not another service database.

5. Functional Requirements

5.1 Bag and journey lifecycle

- Register a bag with unique bagTag, pseudonymous passenger reference, itinerary ID, creation timestamp and route legs.
- Validate airport codes, connecting-leg chronology and planned transfer time; duplicate bagTag registration returns HTTP 409.
- Accept scan observations with eventId, bagTag, checkpoint, airport, device timestamp, ingest timestamp and device sequence.
- Display both latest accepted journey state and complete immutable raw-event audit history.

5.2 Connection risk and anomaly detection

- Identify bags with less than 20 minutes to departure and no aircraft-loading scan at the correct gate; threshold configurable by airport.
- Identify impossible transitions (e.g., ARRIVED at a different airport before an eligible flight leg) without destroying raw input history.
- A repeated eventId has no additional business effect, even across Kafka retries or process restarts.
- Out-of-order observations within a configured grace period must update projections by event-time rules; older events remain auditable.

5.3 Recovery planning and capacity

- Open one active recovery case per bag plus itinerary-leg pair; competing requests cannot create duplicates.

- Recovery states: OPEN -> ASSESSING -> RESERVED -> IN_TRANSIT -> RESOLVED. CLOSED_AS_UNRECOVERABLE and CANCELLED are terminal alternatives; invalid moves return 409.
- Choose recovery action based on airline policy, remaining time and available transport capacity: expedited transfer, reroute to next flight, hold at service desk.
- Reserve a recovery vehicle capacity slot atomically; concurrent reservations must never exceed physical capacity.
- If reservation cannot complete before timeout, free any previously held capacity via compensating workflow.

5.4 Search and reporting

- Search by bagTag, airport, airline, flight, risk category and case state with paging and deterministic sorting.
- Publish hourly counts for risky bags, resolved recoveries, retry volumes and mean case resolution duration.
- Provide eventual-consistency lag for the Elasticsearch projection; rebuilding an index must not alter source records.

6. Explicit Business Rules

Rule	Acceptance criterion
BR-01	A bag may have at most one active recovery case per itinerary leg.
BR-02	Risk is HIGH when departure is within 20 minutes and expected load confirmation is absent.
BR-03	Three rejected transitions for one bag in a 5-minute event-time window produce a SCANNER_ANOMALY event.
BR-04	Capacity reservation cannot exceed remaining slots, including with 50 parallel requests.
BR-05	Recovery state moves only along the defined state machine; each move records who/when/why.
BR-06	An event older than the grace interval is preserved for audit and flagged LATE_EVENT.
BR-07	A resolved case cannot be reopened; a new leg requires a new case key.
BR-08	Notification is unique by (recipient, eventId, template), regardless of delivery retries.

7. REST API Contract (Examples)

All public APIs are exposed through the gateway under /api/v1. Use RFC 7807-style ProblemDetail responses and an X-Correlation-Id header. Use Idempotency-Key for mutation retries where appropriate.

```
POST /api/v1/bags
Authorization: Bearer <jwt>
{ "bagTag":"AB123456", "passengerRef":"P-901",
  "itineraryId":"IT-100", "legs":[
    {"flight":"AT401","from":"IST","to":"FRA",
     "departureAt":"2026-11-03T09:00:00Z"},
    {"flight":"AT990","from":"FRA","to":"AMS",
     "departureAt":"2026-11-03T11:05:00Z"}] }

201 Created
{ "bagId":"uuid", "bagTag":"AB123456", "status":"REGISTERED" }

POST /api/v1/scans
Idempotency-Key: 96d0a2d1
```

```
{ "eventId":"EV-1001", "bagTag":"AB123456",
  "airport":"FRA", "checkpoint":"TRANSFER_SORTER",
  "deviceSequence":37, "occurredAt":"2026-11-03T10:48:00Z" }

202 Accepted
{ "eventId":"EV-1001", "ingestionStatus":"QUEUED" }

POST /api/v1/recovery-cases
{ "bagTag":"AB123456", "legId":"LEG-2",
  "reason":"HIGH_CONNECTION_RISK" }

201 Created
{ "caseId":"uuid", "state":"OPEN", "version":1 }

PATCH /api/v1/recovery-cases/{id}/transitions
{ "targetState":"RESERVED", "expectedVersion":2,
  "reason":"Transport slot secured" }
```

Also implement GET /bags/{bagTag}/journey, GET /recovery-cases?state=OPEN&page=0&size=20, POST /capacity/reservations, DELETE /capacity/reservations/{id}, and GET /reports/airport/{iata}?from=&to=. Document success, validation, conflict and forbidden responses in OpenAPI.

8. Kafka Event Contracts and Streams Topology

Topic	Key / producer / main consumer
baggage.scan.received.v1	bagTag / journey-service / risk-streams-service
baggage.scan.anomaly.v1	bagTag / risk-streams-service / recovery-service
baggage.risk.changed.v1	bagTag:legId / risk-streams-service / recovery-service
recovery.case.changed.v1	caseId / recovery-service / search-service, notification-service
recovery.capacity.changed.v1	resourceId / capacity-service / recovery-service
baggage.dead-letter.v1	eventId / failed-event handler / operator investigation

- Use versioned JSON event envelopes: eventId, eventType, schemaVersion, aggregateId, occurredAt, publishedAt, correlationId, payload.
- Use event-time tumbling or hopping windows for rejected-transition counts; document grace, suppression and window boundary behavior.
- Use a state store / materialized table for latest legitimate checkpoint per bag; partition by stable bagTag.
- Prove idempotent business effects with durable deduplication; Kafka exactly-once processing does not replace end-to-end idempotency across databases.
- Define retry, dead-letter and poison-message policies. Provide reproducible late, duplicate and out-of-order event fixtures.

9. Database Design and Liquibase Migration

- Model baggage, itinerary, itinerary_leg, scan_audit, recovery_case, state_transition, capacity_resource, capacity_reservation, outbox_event and processed_event tables.
- Add unique constraints on bag_tag, source event_id and active recovery case natural key; use optimistic locking (version) for state transitions.

- V1: base tables and seed data. V2: add nullable scan_source and backfill existing rows. V3: enforce not-null only after validation, with a documented rollback strategy.
- Perform a zero-data-loss migration demonstration: create legacy rows, apply V2/V3, compare checksums and row counts; explain rollback limits for destructive changes.
- Use one Liquibase changelog per schema/service; migrations must be safe to re-run and include index choices.

10. Distributed Communication and Design Patterns

- Strategy: select recovery action based on urgency, route options and configurable airline rules.
- Factory Method: construct versioned incoming scan handlers by scanner protocol.
- State: enforce recovery state transitions with explicit guard conditions.
- Adapter: normalize simulated barcode, RFID and conveyor checkpoint payload formats.
- Observer / domain events: decouple state changes from notification intents.
- Transactional Outbox: atomically persist a case change and outbound event; relay via scheduled publisher with at-least-once delivery.
- Saga/compensation: cancel capacity holds if a recovery operation cannot complete; document failures between steps.
- Use typed @HttpExchange clients between recovery-service and capacity-service, configured with timeouts, correlation propagation and explicit fallback behavior.

11. Security and Audit

- Issue signed JWT access tokens with roles PASSENGER, AGENT, DISPATCHER and ADMIN; validate signature, expiration, issuer and audience at gateway and resource services.
- Enforce row-level ownership: a passenger may only view baggage explicitly bound to their pseudonymous account.
- Only agents/dispatchers may transition recovery cases; only admins may change risk policies or manage capacity.
- Use HTTP 401 for missing/invalid token, 403 for insufficient permission. Never log JWTs or raw passenger data.
- Keep immutable audit trail for security-relevant and operational state changes. Document secret management for Minikube.

12. Redis and Elasticsearch

- Redis caches read-only flight lookup and risk policy data with explicit TTL and invalidation after policy changes; database remains authoritative.
- Redis may be unavailable without permitting duplicate case creation or overbooking; document degraded-mode behavior.
- Elasticsearch indexes baggage journey and recovery case documents, supports filtered search and search_after pagination for deep pages.
- Create an idempotent indexer and a full backfill/reconciliation command; measure and expose indexing lag.

13. Local Runtime and Kubernetes Delivery

- Provide Dockerfiles and a docker-compose.yml that launches PostgreSQL, Kafka, Redis, Elasticsearch, Eureka, gateway and the services with usable health checks.
- Provide Kubernetes YAML or Helm charts deployable to Minikube, including ConfigMaps, Secrets, Services, Deployments, PVCs where needed, readiness/liveness probes and resource requests/limits.
- Demonstrate two journey-service replicas, and ensure the application remains correct during pod restarts and a rolling update.
- Document port-forwarding and service discovery; explain how service discovery and gateway routing work in both Compose and Minikube.

14. Observability: Prometheus, Grafana, Loki and Tempo

- Expose Micrometer metrics: scan_ingest_total, scan_duplicates_total, baggage_risk_high, recovery_resolution_seconds, capacity_reservation_conflicts_total, stream_lag_seconds, outbox_pending_count.
- Provide a Grafana dashboard with scan throughput, high-risk bags, case state distribution, P95 API latency, error rate, Kafka consumer lag and JVM health.
- Export structured JSON logs to Loki carrying traceId, spanId, correlationId, service and eventId (no passenger PII).
- Instrument gateway-to-service HTTP calls and Kafka consumers/producers using OpenTelemetry; export traces to Tempo and link Grafana logs to traces.
- Create actionable example alerts: high-risk bag surge, outbox backlog, prolonged stream lag and reservation conflicts.

15. Testing Requirements

- JUnit 5 + Mockito Given/When/Then unit tests for policy strategies, state guards, validation, idempotency and compensations.
- Spring Boot integration tests using Testcontainers for PostgreSQL, Kafka and Redis; use Elasticsearch test container where indexing is verified.
- Verify Kafka Streams with deterministic event-time and topology fixtures; test boundaries around the grace period.
- Test security authorization matrix, gateway token forwarding and @HttpExchange error/timeout behavior.
- Provide race-condition tests with at least 50 simultaneous requests attempting to reserve 10 available slots.
- Minimum 80% line coverage on domain/application code; no test or generated code counted toward the threshold.

16. GitHub Actions CI/CD and SonarQube

- On pull request: build with Java 25, validate formatting, execute unit and Testcontainers integration tests, publish JUnit results and coverage, run SonarQube quality gate and build Docker images.
- On main after quality gate: tag versioned images and publish to a configurable registry; deploy to Minikube via a documented self-hosted runner workflow (or provide a fully executable manual deployment when runner access is absent).

- Scan dependencies and container images; no secrets in repository or logs; never deploy failing or untested images.
- Include a rollback recipe to previous image tag and a post-deployment smoke test of gateway, scan processing and Grafana metrics.

17. Non-Functional Targets

Target	Requirement
Correctness	No duplicated active recovery case; no capacity overbooking; traceable state transitions
Performance	Under simulated local load: 100 scans/s sustained for 2 minutes; P95 read API < 400 ms after warm-up
Resilience	No lost committed outbox event during Kafka outage/restart; eventual replay after recovery
Observability	All public write paths traceable with correlated logs and metrics
Security	RBAC enforced, PII masked, JWT validation on all protected resource paths

Performance figures are case-study targets, not production SLAs. Record the computer configuration and test methodology with each benchmark.

18. Acceptance Test Scenarios

- AT-01 Register a valid baggage journey and retrieve its ordered itinerary legs.
- AT-02 Reject an existing bagTag with a stable 409 error contract.
- AT-03 Submit an event twice and observe one effective journey change.
- AT-04 Apply a late-but-within-grace scan by event time, not arrival order.
- AT-05 Preserve an excessively late scan in audit and flag LATE_EVENT.
- AT-06 Detect three invalid transitions in a five-minute event-time window.
- AT-07 Trigger HIGH risk when an unconfirmed connecting leg departs in under 20 minutes.
- AT-08 Clear or downgrade risk on a valid loading confirmation; no false extra case.
- AT-09 Create one active case from two parallel attempts for the same bag and leg.
- AT-10 Reject an illegal transition from RESOLVED to IN_TRANSIT.
- AT-11 Reserve only 10 slots when 50 requests arrive concurrently.
- AT-12 Release a provisional capacity hold after a recovery timeout.
- AT-13 Repeat outbox publication and observe one effective notification.
- AT-14 Stop Kafka during a case transition; replay the committed event on restart.
- AT-15 Block a passenger querying another passenger's bag.
- AT-16 Prove an expired JWT returns 401 and an unauthorized role returns 403.
- AT-17 Apply V1-V3 Liquibase migrations to preloaded data without row loss.
- AT-18 Rebuild Elasticsearch from source data with no duplicate documents.
- AT-19 Take Redis offline and verify no incorrect capacity reservation.
- AT-20 Propagate a trace through gateway, journey-service and Kafka consumer into Tempo.
- AT-21 Collect a service log in Loki and correlate it to the matching Tempo trace.

- AT-22 Restart a Minikube pod without losing committed recovery state.
- AT-23 Fail SonarQube quality gate and confirm no image deployment occurs.
- AT-24 Demonstrate deployment rollback and gateway smoke tests.

19. Submission Artifacts

- Git repository with README, architecture and sequence diagrams, Docker Compose, Minikube manifests, Liquibase scripts, sample seed data and Postman collection.
- OpenAPI specifications with authorization and error response examples; curl examples for the complete happy path and failure paths.
- Automated test suite, coverage report, concurrency test output and example Kafka replay dataset.
- GitHub Actions workflow and SonarQube configuration; include screenshots or exported Grafana dashboards and tracing evidence.
- Architecture Decision Records documenting state management, exactly-once business effects, Redis fallback and data migration strategy.

20. Evaluation Rubric (100 points)

Category	Points
Domain correctness and concurrency	25
Microservice boundaries and design patterns	15
Kafka Streams, retries and outbox	15
Data modeling, Liquibase and search	12
Tests and security	13
Docker, Minikube and CI/CD	10
Observability and documentation	10

21. Optional Advanced Challenges

- Model a missed connection across a two-leg itinerary where airport clocks are misconfigured; retain provenance and reject impossible chronology.
- Implement multi-tenant airline isolation at gateway, service and search index levels.
- Enable blue/green migration of an event payload from schema v1 to v2 without interrupting existing consumers.
- Run a chaos experiment (Kafka unavailable or one database replica restarted), document recovery timelines and integrity checks.

22. Notes for Candidates

Assumptions are permitted if explicit, testable and documented. Prefer clear business semantics, stable API contracts, reproducible automation and evidence of correctness over the number of endpoints. Substituting mock services is acceptable only where stated (airline feeds and notification provider); required infrastructure technologies must be exercised by tests or runtime demonstrations.

Definition of done: A reviewer can clone the repository, launch the development stack, seed a baggage journey, produce duplicate and late scans, observe a high-risk case, reserve constrained capacity, query results, inspect metrics/logs/traces, execute tests and reproduce the delivery pipeline.